

Developing Agentic Systems: 10 Lessons from the Overwatch Harness & the Open-Source Ecosystem

Primary Reference: [ivenkwan/Overwatch](#) — AML Platform (Unified TradFi & Web3 Fund Flow Analysis Engine)

Supplementary Reference: Open-source agentic frameworks — LangChain/LangGraph, CrewAI, AutoGen, Dagster, Flowable

Analysis Date: June 2026

What Is an Agentic Harness?

An agentic harness is the foundational infrastructure that surrounds a core intelligence — whether an LLM, a rules engine, or a hybrid — and transforms it from a passive processor into a functional, autonomous system capable of sustained, multi-step work. The harness provides the essential scaffolding: tool usage, memory, state transitions, safety boundaries, orchestration, and human intervention points. The LLM (or rule engine) provides the "brain"; the harness provides the "body."

The equation to internalise is: **Agent = Intelligence + Harness**. The Overwatch repository demonstrates this with unusual clarity because its "intelligence" is not an LLM — it is a set of AML typology rules, Cypher graph queries, and a BPMN workflow engine. This is instructive: the agentic harness pattern applies equally to rule-based, ML-based, and LLM-based systems. The architecture lessons generalise across all three.

The repository spans six integrated services:

- **Next.js frontend** — investigation workspace and visual graph explorer
- **Python/FastAPI backend** — API layer, RBAC, and service orchestration
- **PostgreSQL + Apache AGE** — hybrid relational and graph database
- **Dagster ETL pipeline** — batch data ingestion and graph promotion
- **Flowable BPMN engine** — human-in-the-loop case approval workflow
- **Keycloak** — identity and access management

Lesson 1: The Harness Is as Critical as the Intelligence

Principle: A common misconception is that an agent's power lies solely in its core model or rule engine. In practice, the harness — the collection of code, configuration, and execution logic that envelops the intelligence — is what translates raw capability into useful, reliable work.

From Overwatch: The platform's AML detection capability is not sophisticated in isolation: it runs Cypher graph queries for circular flow, smurfing, and mule detection. What makes it powerful is the harness around it — the ETL pipeline that prepares data, the RBAC system that

gates access, the Flowable engine that manages approvals, and the audit chain that makes every decision legally defensible. Strip out the harness, and the Cypher queries become a standalone script with no operational value.

From the ecosystem: LangChain, CrewAI, and AutoGen all demonstrate the same principle from the LLM angle — the same GPT-4 model produces dramatically different outcomes depending on how the surrounding harness is designed. Tool selection, prompt structure, memory management, and error handling all live in the harness, not in the model. The `.agents/rules/graphify.md` file in Overwatch further encodes this philosophy: it instructs the coding agent on how to navigate the codebase, separating "what the agent knows" from "how the agent is governed."

Takeaway: Invest in harness design with the same rigour as intelligence design. A well-engineered harness multiplies the value of any intelligence behind it; a poorly engineered one caps it.

Lesson 2: Decompose the Harness into Modular, Extensible Components

Principle: A harness should not be a monolithic block of code. Breaking it into distinct, replaceable components enables independent testing, debugging, and evolution — and prevents the "spaghetti agent" anti-pattern where everything is entangled.

From Overwatch: The repository's layered N-Tier architecture is a direct application of this lesson. The harness is decomposed into:

- `app/core/` — authentication, RBAC, and audit primitives
- `app/db/` — database connection pooling and session management
- `app/services/` — domain services (`flowable_client.py`, `graph_service.py`, `pii_service.py`)
- `app/api/v1/` — HTTP endpoint routers (`alerts.py`, `cases.py`, `graph.py`, `reports.py`, `admin.py`)

Each component has a single responsibility. `pii_service.py` only masks data; it knows nothing about authentication. `flowable_client.py` only communicates with the Flowable engine; it knows nothing about the database schema. This separation means that when the PII masking rules change (e.g., adding email as a sensitive field), only one file changes.

From the ecosystem: LangGraph formalises this decomposition into three primitives — **State** (shared data structure), **Nodes** (worker functions), and **Edges** (routing logic). CrewAI decomposes into **Agents** (role-defined workers), **Tasks** (assignments with expected outputs), and **Crews** (orchestrated teams). AutoGen uses **ConversableAgent**, **AssistantAgent**, and **UserProxyAgent** as composable building blocks. In all cases, swapping or extending one component does not require rewriting the others.

Takeaway: Design every harness layer as a replaceable module. Use dependency injection (FastAPI's `Depends()` pattern in Overwatch) to wire components together without hard-coding dependencies.

Lesson 3: Idempotency Is Non-Negotiable for Reliable Agents

Principle: Every agentic operation that modifies persistent state must be idempotent — safe to retry without creating duplicate or inconsistent records. Agents fail and restart; idempotency is the guarantee that a retry is indistinguishable from a first run.

From Overwatch: The ETL pipeline in `daily_pipeline.py` is a textbook implementation. Each transaction is assigned a deterministic content-addressed key — a SHA-256 hash of `customer_num + txn_date + txn_ref_no + txn_currency_amount` — so the same transaction always produces the same hash regardless of how many times it is processed. Every INSERT statement uses `ON CONFLICT ... DO NOTHING`:

```
INSERT INTO core.transactions (...) VALUES %s ON CONFLICT (txn_hash) DO NOTHING
INSERT INTO core.customers (customer_num) VALUES %s ON CONFLICT (customer_num) DO NOTHING
INSERT INTO core.counterparties (...) VALUES %s ON CONFLICT (counterparty_id) DO NOTHING
```

The graph model update uses Cypher `MERGE` instead of `CREATE` to prevent duplicate nodes and edges. The Keycloak user provisioning in `admin.py` uses `ON CONFLICT (keycloak_user_id) DO UPDATE SET email = EXCLUDED.email`.

From the ecosystem: LangGraph's checkpointers achieve idempotency at the workflow level — re-running from a checkpoint replays only the steps that haven't completed, preventing duplicate side effects. Dagster's `run_key` in the daily schedule (`run_key=context.scheduled_execution_time.strftime("%Y%m%d")`) prevents the same day's job from running twice.

Takeaway: Design agents around natural keys and content hashes, not auto-incremented IDs. Use UPSERT semantics at every persistence boundary. A re-runnable agent is a resilient agent.

Lesson 4: Use a Dead Letter Queue to Prevent Silent Data Loss

Principle: Agents that process external data must handle partial failures gracefully. Errors in individual records must not silently discard data, cascade into full pipeline crashes, or leave the system in an ambiguous state.

From Overwatch: The Dagster pipeline implements a two-tier DLQ pattern. At the **batch level**, the `rename_to_ok` op renames the source file to `{filename}.OK` only after all five upstream ops have completed successfully. If `verify_data_load` raises a `Failure` (e.g., because the loaded count is zero), the rename never executes — the file stays in the input directory and remains available for manual reprocessing or the next scheduled run. At the **row level**, the `load_csv_to_postgres` op calls `.drop_nulls(subset=["txn_ref_no"])` before insertion, filtering bad rows defensively rather than crashing on them. The `AML_spec.md` explicitly lists a "Dead Letter Queue (DLQ) for failed data ingestion triage" as a named operational requirement.

From the ecosystem: This mirrors Dagster's native failure handling via the `Failure` exception (which marks a run as failed without crashing the scheduler) and LangGraph's error nodes (which route exceptions to a dedicated error-handling branch rather than propagating them up).

Takeaway: Build two DLQ tiers into every agentic pipeline: a **batch-level** DLQ (file markers, queue acknowledgements, or run status flags) that prevents source data deletion until success is confirmed, and a **row-level** DLQ (validation filters or error tables) that captures bad records without stopping the pipeline.

Lesson 5: Orchestrate with Dual-Layer State — Workflow Engine + Application Database

Principle: In a compound agentic system, state must be tracked simultaneously in two stores: the workflow engine (orchestration truth — what step is active, who is responsible next) and the application database (query truth — what is the current status for dashboards and APIs). Neither alone is sufficient.

From Overwatch: The `cases.py` endpoint is the clearest demonstration of this pattern. The `action_case` endpoint performs a **dual-write** on every state transition:

```
# Step 1: Advance the BPMN state machine
await flowable_client.complete_task(active_task["id"], variables=flowable_vars)

# Step 2: Sync the relational database
update_query = "UPDATE app.cases SET status = $1 WHERE case_id = $2"
await db.execute(update_query, new_db_status, case_id)
```

When fetching case details, both stores are read and merged:

```
case_info = dict(row) # From PostgreSQL
active_task = await flowable_client.get_active_task(workflow_instance_id) # From Flowable
case_info["activeTask"] = active_task # Merged response
```

The BPMN definition (`aml_case_workflow.bpmn20.xml`) encodes the maker-checker approval logic with an `exclusiveGateway` that routes to either `endApproved` or back to `makerTask` on rejection.

From the ecosystem: LangGraph's checkpoints implement the same dual-layer concept — the graph state (edges, node history) is the "workflow truth" stored in `SqliteSaver`, while the application database holds the business truth. AutoGen's GroupChat history is the workflow truth; any downstream persistence is the application truth.

Takeaway: Do not collapse workflow state and application state into a single store. Design explicit synchronisation points at every state transition, and build read endpoints that merge both stores into a unified view.

Lesson 6: Compliance-First Design — Bake Security Into the Foundation

Principle: In regulated or sensitive domains, compliance features (audit trails, RBAC, PII masking) are not add-ons to be retrofitted. They must be woven into the data model, the database schema, and the API from the very first commit.

From Overwatch: Three separate layers enforce compliance, each independent of the others:

- **Database-level RBAC with Row-Level Security:** `01_rbac_initialization.sql` creates two segregated roles — `aml_api_role` (cannot bypass RLS) and `aml_etl_role` (bypasses RLS for bulk loads). RLS policies enforce tenant isolation using `current_setting('app.current_tenant', true)`.
- **Tamper-evident cryptographic audit chain:** `02_audit_tamper_evidence.sql` installs a `BEFORE INSERT` trigger that computes a SHA-256 hash of each row chained to the `previous_hash` of the prior record — making retroactive log manipulation cryptographically detectable. A second `BEFORE UPDATE OR DELETE` trigger raises an exception on any modification, making the audit table physically append-only.
- **Role-gated PII masking at the API layer:** `pii_service.py` recursively traverses all API response payloads and masks wallet addresses, account numbers, and personal identifiers based on the caller's role. Only `SENIOR_INVESTIGATOR` receives unmasked data, and every such access triggers `log_unmasking_event()` which writes to the audit trail.

From the ecosystem: LangGraph's `interrupt_before` mechanism (pausing before tool execution for human approval) and AutoGen's `UserProxyAgent` with `code_execution_config={"use_docker": True}` (sandboxed execution) are harness-level safety controls that parallel Overwatch's approach — neither requires changes to the core intelligence to enforce.

Takeaway: Implement compliance at the infrastructure level (DB triggers, RLS), the service level (PII middleware), and the API level (scope-enforced decorators) simultaneously. Relying on a single enforcement point creates a single point of failure in a compliance audit.

Lesson 7: Externalise Agent Configuration — Adopt the `.agents/` Convention

Principle: Agentic harnesses require a dedicated configuration layer that governs how agents navigate, reason about, and modify the system. This configuration should be version-controlled alongside the code and separated from application logic.

From Overwatch: The repository introduces a `.agents/` directory at the project root as a first-class part of the codebase. The `.agents/rules/graphify.md` file contains behavioural rules for the coding agent:

```
"Before answering architecture or codebase questions, read graphify-  
out/GRAPH_REPORT.md for god nodes and community structure"  
"After modifying code files in this session, run graphify update . to keep the graph  
current (AST-only, no API cost)"
```

The `.agents/workflows/` directory separates workflow-specific procedures from general rules. This externalises agent policy from application code — changing the agent's behaviour requires

only editing a markdown file, not modifying any Python or SQL.

From the ecosystem: This maps directly to the concept of **system prompts** and **agent configuration files** in LangChain (`.langchain/ configs`), CrewAI (YAML-based agent and task definitions), and AURA (declarative TOML workflow definitions). The `.agents/` convention is a generalisation of this principle to any codebase, regardless of whether the agent is LLM-based or rule-based.

Takeaway: Create a `.agents/` directory in every agentic project on day one. Separate **rules** (invariants the agent always follows) from **workflows** (task-specific procedures). Treat these files with the same version-control discipline as application code — they are the operating manual for your agent.

Lesson 8: Invest in Codebase Cartography — Build a Machine-Readable Map

Principle: As an agentic codebase grows beyond a few hundred files, agents and developers alike need a structured, queryable map of the system. Raw file trees and `grep` are insufficient for understanding cross-module dependencies, identifying high-risk components, and surfacing undocumented connections.

From Overwatch: The `graphify-out/` directory contains the output of the `graphify` knowledge graph tool applied to the entire codebase. The `GRAPH_REPORT.md` shows:

- **201 nodes, 188 edges, 21 communities** extracted from 121 files (~169,902 words)
- **God Nodes** (highest-risk abstractions): `daily_update_job()` (6 edges), `mask_pii()` (5 edges), `get_auth()` (5 edges), `run_graph_promotion()` (5 edges) — these are the components where a breaking change has the widest blast radius
- **Knowledge Gaps:** 27 isolated nodes with ≤ 1 connection — likely undocumented components or missing interfaces
- **Community cohesion scores:** `Community 19` (score 1.0 — tightly coupled) vs `Community 0` (score 0.11 — a candidate for splitting)

The `.agents/rules/graphify.md` instructs the coding agent to consult this graph report before answering any architecture question — prioritising graph-based navigation over brute-force file search.

From the ecosystem: LangSmith's tracing and evaluation for LangChain agents, and AutoGen's conversation logging, serve the same cartographic purpose at runtime — they build a map of what the agent did, which tools it called, and where it failed. Codebase cartography (static) and execution cartography (runtime) are complementary.

Takeaway: Run a codebase knowledge graph tool (`graphify`, `Sourcegraph`, or similar) as part of your CI pipeline and commit the output to version control. God Nodes reveal your most fragile abstractions; Knowledge Gaps reveal your documentation blind spots. Make the graph queryable by your agents.

Lesson 9: Design for Hybrid Data Topology — Match Storage to Query Pattern

Principle: Complex agentic tasks require multiple data access patterns. An agent that only queries relational tables cannot efficiently answer "who are the second-degree counterparties of this customer?" — and a pure graph database cannot efficiently answer "what is the total HKD value of all debit transactions this month?" Design the data layer to serve both.

From Overwatch: The central architectural innovation is running Apache AGE (an openCypher graph extension) on top of PostgreSQL — providing both SQL and graph query capabilities from a single database connection. The `session.py` establishes dual mode on every connection:

```
await conn.execute("LOAD 'age';")
await conn.execute("SET search_path = ag_catalog, \"$user\", public;")
```

The ETL pipeline performs the "dual-write": `load_csv_to_postgres` populates `core.transactions` (relational), and `update_graph_model` immediately projects that same data into the AGE graph using Cypher MERGE statements, creating Customer, Merchant, and Counterparty nodes. The `graph_service.py` implements neighborhood expansion — fetching a subgraph up to 6 hops deep from any entity — which is trivially expressed in Cypher but would require complex recursive SQL.

From the ecosystem: LangGraph's `SqliteSaver` (relational) and vector store memory (semantic retrieval) demonstrate the same multi-topology principle for LLM agents. Different query types — "what happened in step 3?" (relational), "what concepts are similar to X?" (vector), "who is connected to Y?" (graph) — require different storage primitives. The harness must wire all of them together.

Takeaway: For investigative, fraud-detection, or network-analysis use cases, choose a hybrid data topology from the start. Use relational tables as the operational source of truth and graph databases as the analytical projection. Run the "graph promotion" step as a mandatory final stage of every ETL pipeline.

Lesson 10: Practise Incremental Honesty — Track Debt, Gaps, and Shortcuts Explicitly

Principle: An agentic harness is never complete. The difference between a healthy codebase and a deteriorating one is not the absence of shortcuts — it is whether those shortcuts are documented, owned, and scheduled for resolution. Both agents and human developers need this transparency to make safe changes.

From Overwatch: The `todo.md` committed to the repository root provides a granular [DONE] / [] checklist across frontend, backend, and fix categories. It documents known technical debt honestly:

- [] Implement frontend authentication with auth headers on API calls — currently, the frontend does not attach JWT tokens, meaning RBAC is only enforced server-side.

- [] Persist audit logs to database or tamper-evident log sink — `log_audit_event()` in `auth.py` currently writes to a Python logger, not to the `app.audit_access_events` table that was carefully designed in `02_audit_tamper_evidence.sql`. This gap between the intended and actual audit sink is a compliance risk that the `todo.md` makes explicit and visible.

A `SECURITY_SCAN_20260403.md` file records a dated security audit, creating a lightweight compliance paper trail that travels with the code. The `cases.py` comment — "For now, generate a random tenant manually as we don't handle cross-schema perfectly here" — is a direct in-code annotation of a known shortcut.

From the ecosystem: This principle maps to the broader open-source practice of issue tracking, RFC documents, and `FIXME` annotations. Frameworks like `LangChain` maintain explicit migration guides and deprecation notices. The `Overwatch` `todo.md` pattern is a leaner, repo-local version of the same discipline.

Takeaway: Commit a `todo.md` to every agentic project and update it with every sprint. Distinguish between **permanent design decisions**, **deferred work**, and **known shortcuts**. Pair it with dated security scan reports. A codebase that documents its gaps honestly is safer to operate than one that silently contains the same gaps but hides them.

Cross-Reference: Overwatch ↔ Open-Source Ecosystem

The following table maps each lesson to its `Overwatch` artefact and the equivalent pattern in the broader agentic framework ecosystem:

#	Lesson	Overwatch Artefact	Ecosystem Equivalent
1	Harness = Critical Infrastructure	<code>main.py</code> lifespan + 6-service stack	<code>LangChain</code> : harness around GPT-4
2	Modular, Extensible Components	<code>core/</code> , <code>db/</code> , <code>services/</code> , <code>api/v1/</code>	<code>LangGraph</code> : <code>State/Node/Edge</code> ; <code>CrewAI</code> : <code>Agent/Task/Crew</code>
3	Idempotency via Content Hashing	SHA-256 <code>txn_hash</code> + <code>ON CONFLICT</code>	Dagster <code>run_key</code> ; <code>LangGraph</code> checkpoint dedup
4	Dead Letter Queue Pattern	<code>rename_to_ok()</code> + <code>drop_nulls()</code>	Dagster <code>Failure</code> ; <code>LangGraph</code> error nodes
5	Dual-Layer State Management	Flowable + PostgreSQL dual-write	<code>LangGraph</code> checkpoint + app DB; <code>AutoGen</code> chat history + persistence
6	Compliance-First Design	RLS + hash chain + PII masking	<code>LangGraph</code> <code>interrupt_before</code> ; <code>AutoGen</code> Docker sandbox
7	Externalised Agent Config	<code>.agents/rules/graphify.md</code>	<code>CrewAI</code> YAML agents; AURA TOML workflows
8	Codebase Cartography	<code>graphify-out/GRAPH_REPORT.md</code>	<code>LangSmith</code> tracing; <code>AutoGen</code> conversation logs
9	Hybrid Data Topology	PostgreSQL + Apache AGE dual-mode	<code>LangGraph</code> : <code>SqliteSaver</code> + vector store memory

#	Lesson	Overwatch Artefact	Ecosystem Equivalent
10	Incremental Honesty	todo.md + SECURITY_SCAN_20260403.md	GitHub Issues, RFC docs, deprecation notices

Framework Selection Guide

For developers choosing an agentic framework, the following decision matrix captures the trade-offs:

Framework	Best For	Key Strength	Watch Out For
LangGraph	Complex, stateful, cyclical workflows	Fine-grained control over graph logic	Steeper learning curve
CrewAI	Rapid, role-based team prototyping	Intuitive role/task abstraction	Less control over low-level state
AutoGen	Conversational, emergent multi-agent systems	Native human-in-the-loop	Less structured for deterministic pipelines
Dagster	Batch data pipelines with observability	Asset-oriented, built-in DLQ	Overkill for simple scripts
Flowable	Human-approval BPMN workflows	Standards-based, auditable	Java dependency, heavier footprint
AURA	Version-controlled, declarative workflows	TOML configs in Git	Smaller community
Mastra	TypeScript/Next.js stacks	Native web integration	Python ecosystem not available

Synthesised Design Principles

The ten lessons converge on five foundational principles for agentic system design:

- 1. Separation of Intelligence and Governance** — the harness governs; the intelligence executes. Keep them in separate modules.
- 2. State as a First-Class Citizen** — model every state transition explicitly, persist it in two stores, and make it queryable.
- 3. Idempotency by Default** — design all writes assuming they will be retried. Use content hashes and UPSERT semantics everywhere.
- 4. Compliance as Infrastructure** — implement audit, RBAC, and data masking at the database layer first, then enforce again at the API layer.
- 5. Honest Documentation** — commit your `todo.md`, your security scan reports, and your architectural decision records. Agents and humans both need them to operate safely.